Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования

ИРКУТСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Институт информационных технологий и анализа данных

наименование института

Отчет по дисциплине

«Методы оптимизации»

по теме:

«Расчет на ЭВМ оптимальных параметров ПИД-регулятора»

Выполнил студент группы	ИСИБ-23-1		Д.С. Привалихин
	Шифр группы	Подпись	И.О. Фамилия
Проверил преподаватель			И.А.Серышева
		Подпись	И.О. Фамилия

Иркутск 2025 г.

Содержание

Введение 3
Ход работы 4
Моделирование замкнутой системы без ПИД-регулятора 6
Моделирование замкнутой системы с ПИД-регулятором 8
Подбор оптимальных параметров ПИД-регулятора методом покоординатного спуска 9
Расчёт функции чувствительности 12
Алгоритм градиентного спуска с нормализацией и адаптивным шагом 14
Алгоритм наискорейшего спуска с линейным поиском ($I_3(2)$) 17
Результаты исследования 39
Вывод 41

Введение

Цель работы — ознакомление с методикой расчета оптимальных параметров регулятора при заданной структурной автоматической системы регулирования и заданных параметрах регулирования по интегральным критериям качества процесса регулирования.

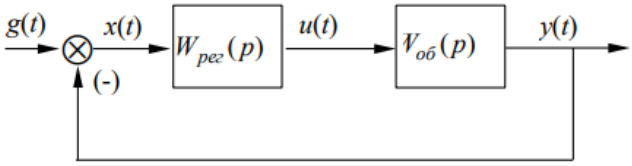

График 1 – Структурная схема системы

Передаточная функция объекта:

$$W_{\text{об}(p)} = \frac{k_{\text{об}}}{T_{\text{об}}^2 p^2 + 2\zeta T_{\text{об}} p + 1} * \lambda^{-\tau_{\text{об}} p}$$

Передаточная функция регулятора:

$$W_{\text{рег}(p)} = q_1 + \frac{q_2}{p} + q_3 p$$

Параметры объекта:

$$k_{\text{об}} = 1, \quad T_{\text{об}} = 1, \quad \zeta = 0,75, \quad \tau_{\text{об}} = 0,1$$

Критерий оптимизации:

$$I_3 = \int_0^{\infty} (x^2(t) + \mu^2 \hat{x}^2(t)) dt$$

Ход работы

$W(p) = \frac{k}{T^2p^2+2\zeta Tp+1}$– колебательное звено

Распишем переход от передаточной функции к дифференциальному уравнению:

$$W(p) = \frac{k}{T^2p^2 + 2\zeta Tp + 1} = \frac{y}{g}$$

$$T^2\frac{d^2y}{dt^2} + 2T\zeta\frac{dy}{dt} + y = kg$$

Избавимся от коэффициента, тогда:

$$\frac{d^2y}{dt^2} + \frac{2\zeta}{T}\frac{dy}{dt} + \frac{y}{T^2} = \frac{k}{T^2}g$$

Коэффициенты для пространства состояний:

$$a_2 = 1, \quad a_1 = \frac{2\zeta}{T}, \quad a_0 = \frac{1}{T^2}, \quad b_0 = \frac{k}{T^2}, \quad b_1 = b_2 = 0, z_3 = 0$$

Подготовим звено к моделированию методом непосредственного интегрирования:

$$y = z_1 + b_2g = z_1$$

$$\frac{dz_1}{dt} = -(a_1y - b_1g) + z_2 = -\frac{2\zeta}{T}y + z_2$$

$$\frac{dz_2}{dt} = -(a_0y - b_0g) + z_3 = -\frac{y}{T^2} + \frac{k}{T^2}g$$

Путем преобразований исходное дифференциальное уравнение второго порядка сводится к эквивалентной системе из двух уравнений первого порядка:

$$\begin{cases} y = z_1 \\ \frac{dz_1}{dt} = -\frac{2\zeta}{T}y + z_2 \\ \frac{dz_2}{dt} = -\frac{y}{T^2} + \frac{k}{T^2}g \end{cases}$$

Применяя метод Рунге-Кутта получаем:

$$\begin{cases} x = g - y \quad y = z_1 \\ k_1 = \mathrm{dt}\ \left(z_2 - \frac{2\zeta}{T}y\right) \\ m_1 = \mathrm{dt}\ \left(\frac{k}{T^2}x - \frac{y}{T^2}\right) \\ k_2 = \mathrm{dt}\ \left(z_2 + \frac{m_1}{2} - \frac{2\zeta}{T}\left(y + \frac{k_1}{2}\right)\right) \\ m_2 = \mathrm{dt}\ \left(\frac{k}{T^2}x - \frac{1}{T^2}\left(y + \frac{k_1}{2}\right)\right) \\ k_3 = \mathrm{dt}\ \left(z_2 + \frac{m_2}{2} - \frac{2\zeta}{T}\left(y + \frac{k_2}{2}\right)\right) \\ m_3 = \mathrm{dt}\ \left(\frac{k}{T^2}x - \frac{1}{T^2}\left(y + \frac{k_2}{2}\right)\right) \\ k_4 = \mathrm{dt}\ \left(z_2 + m_3 - \frac{2\zeta}{T}(y + k_3)\right) \\ m_4 = \mathrm{dt}\ \left(\frac{k}{T^2}x - \frac{1}{T^2}(y + k_3)\right) \end{cases}$$

$$z_1 = z_1 + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

$$z_2 = z_2 + \frac{1}{6}(m_1 + 2m_2 + 2m_3 + m_4)$$

Моделирование замкнутой системы без ПИД-регулятора

График:

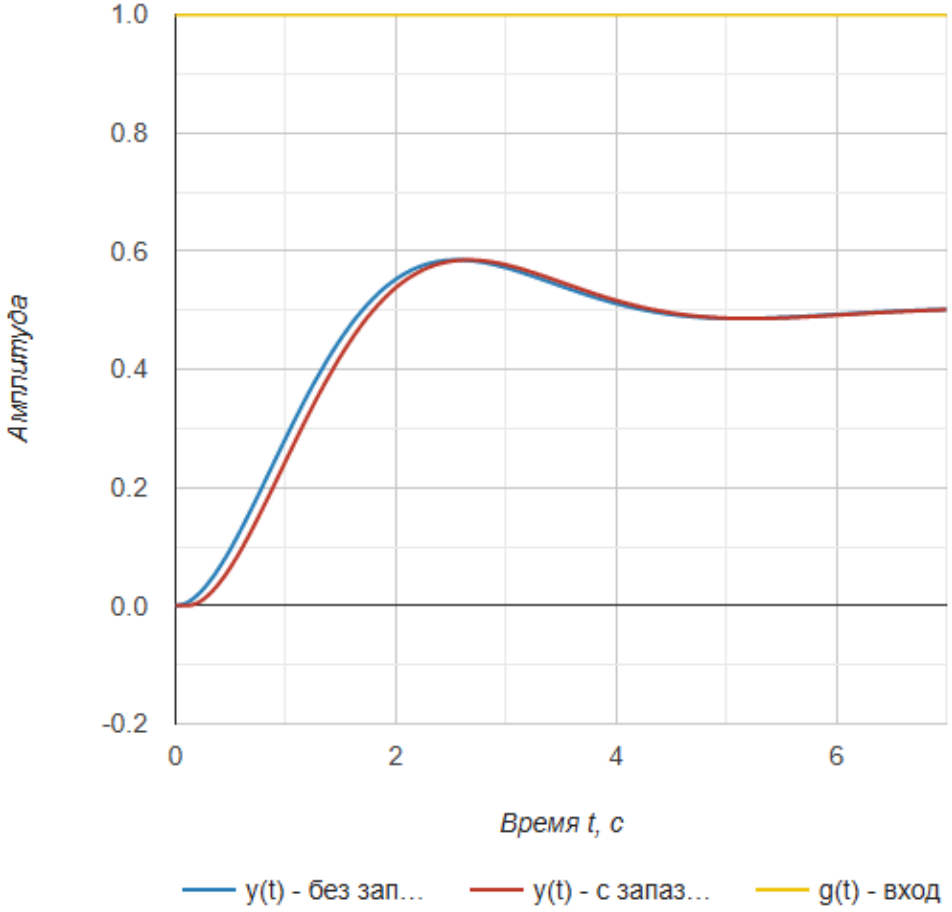

График 2 – Переходный процесс без ПИД-регулятора

Смоделируем данную систему в SimInTech и сравним результаты графиков:
Структурная схема:

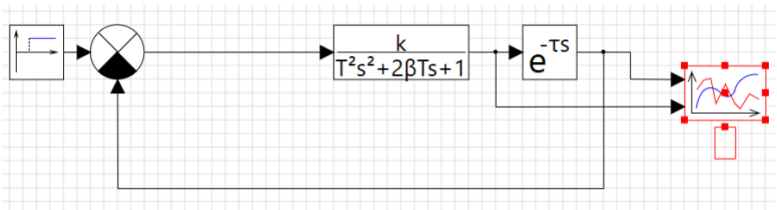

График 3 – Структурная схема без ПИД-регулятора

График:

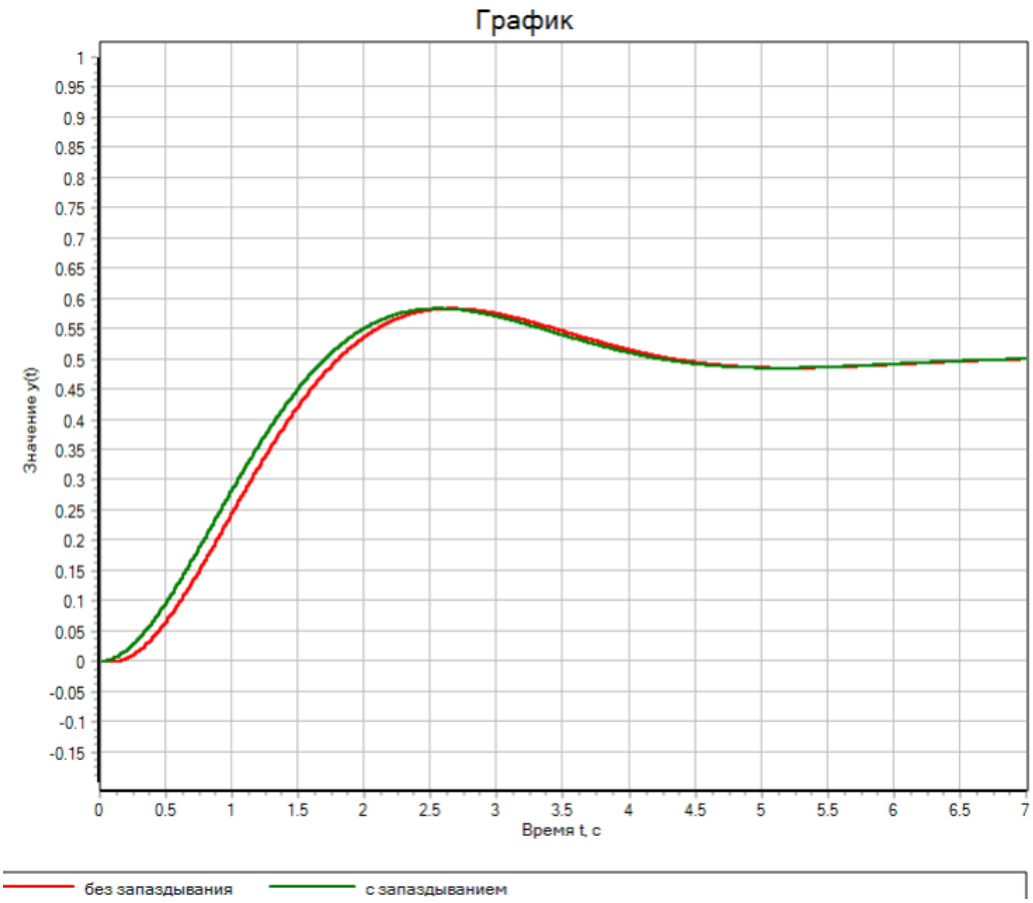

График 4 – Переходный процесс в SimInTech

Как мы видим, графики полностью совпадают, значит наше ручное решение верное.

Моделирование замкнутой системы с ПИД-регулятором

Передаточная функция регулятора:

$$W_{\text{рег}(p)} = q_1 + \frac{q_2}{p} + q_3 p$$

Уравнения системы:

$$u(t,q) = W_{\text{рег}(p)} x(t,q)$$

$$y(t,q) = W_{\text{об}(p)} u(t,q)$$

ПИД-регулятор во временной области:

$$u(t,q) = q_1 x(t,q) + q_2 \int x(t,q)dt + q_3 \frac{dx(t,q)}{dt}$$

Для подбора оптимальных параметров q_1, q_2, q_3 необходим критерий оптимизации. В качестве такого критерия воспользуемся составным интегральным функционалом:

$$I_3 = \int_0^\infty (x^2(t) + \mu^2 \hat{x}^2(t))dt$$

График при параметрах $q_1 = q_2 = q_3 = 1$:

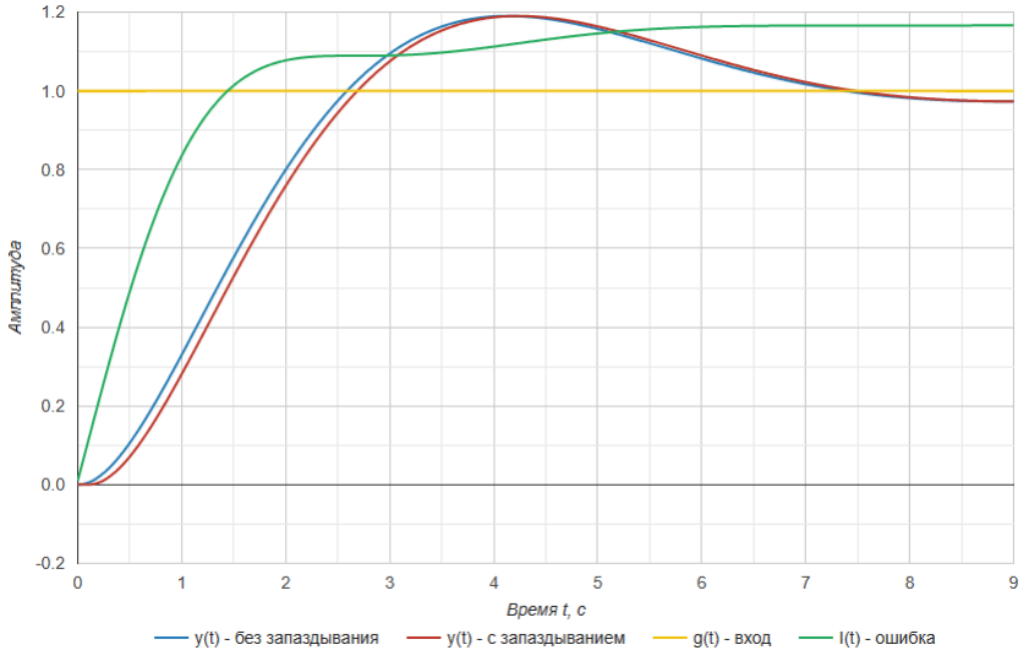

График 5 – Переходный процесс с ПИД-регулятором (начальные параметры)

Итоговое значение интегральной ошибки I: $1,1663$

Подбор оптимальных параметров ПИД-регулятора методом покоординатного спуска

Основная идея метода: Покоординатный спуск — это итерационный метод оптимизации, при котором на каждом шаге оптимизируется только один параметр, а остальные фиксируются. Цикл повторяется до сходимости.

Алгоритм:

1) Инициализировать начальные значения параметров q_1, q_2, q_3 и шаг h.
2) Для каждого параметра q_i $(i = 1, 2, 3)$:
   а) Вычислить значение критерия I при текущих значениях параметров.
   б) Увеличить q_i на шаг h и вычислить новый критерий I_+.
   в) Уменьшить q_i на шаг h (от начального значения) и вычислить критерий I_-.
   г) Если $I_+ < I$ и $I_+ < I_-$, принять $q_i = q_i + h$.
   д) Иначе, если $I_- < I$, принять $q_i = q_i - h$.
3) Если ни один параметр не изменился на шаге 2, уменьшить шаг: $h = \frac{h}{2}$.
4) Повторять шаги 2–3, пока шаг h не станет меньше заданной точности ε.

Найдём оптимальные параметры:

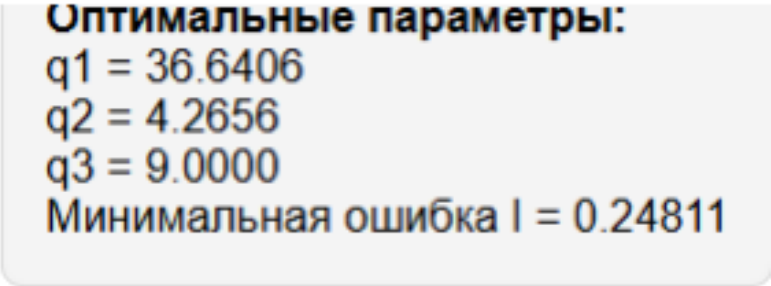

График 6 – Оптимальные параметры ПИД-регулятора

График:

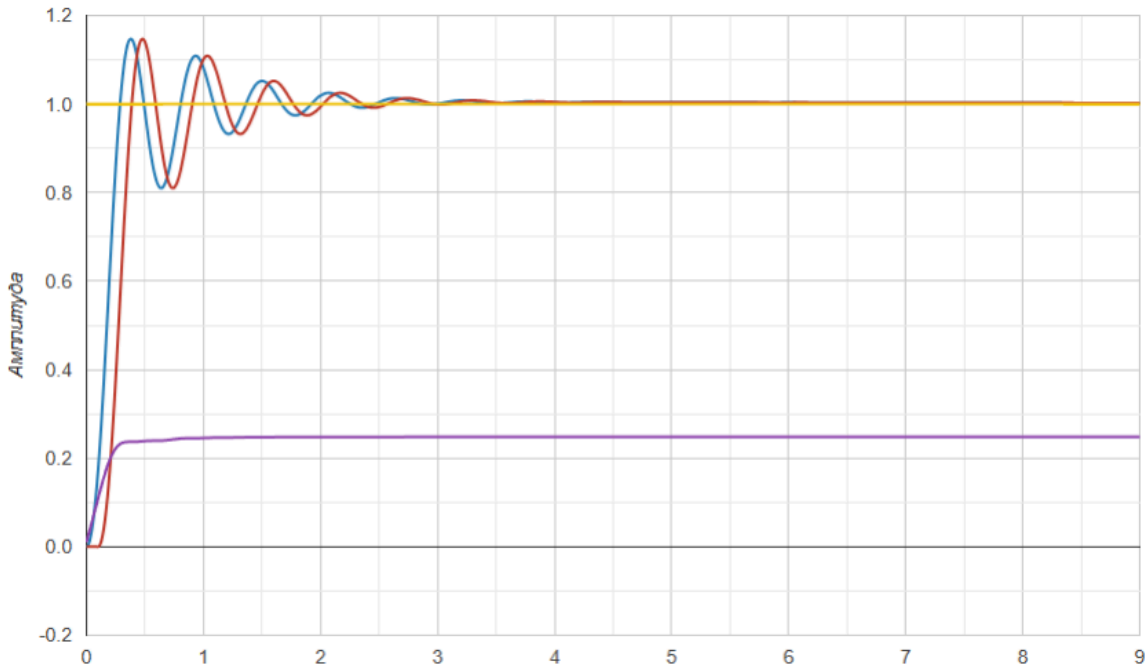

График 7 – Переходный процесс с оптимальными параметрами

Сравним графики с реализацией в SimInTech:
Схема:

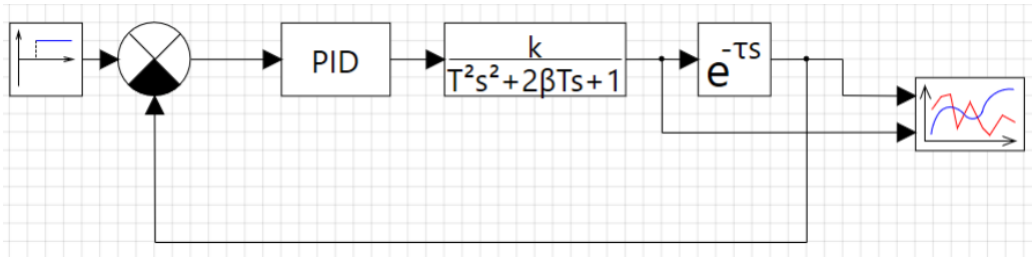

График 8 – Структурная схема в SimInTech

График:

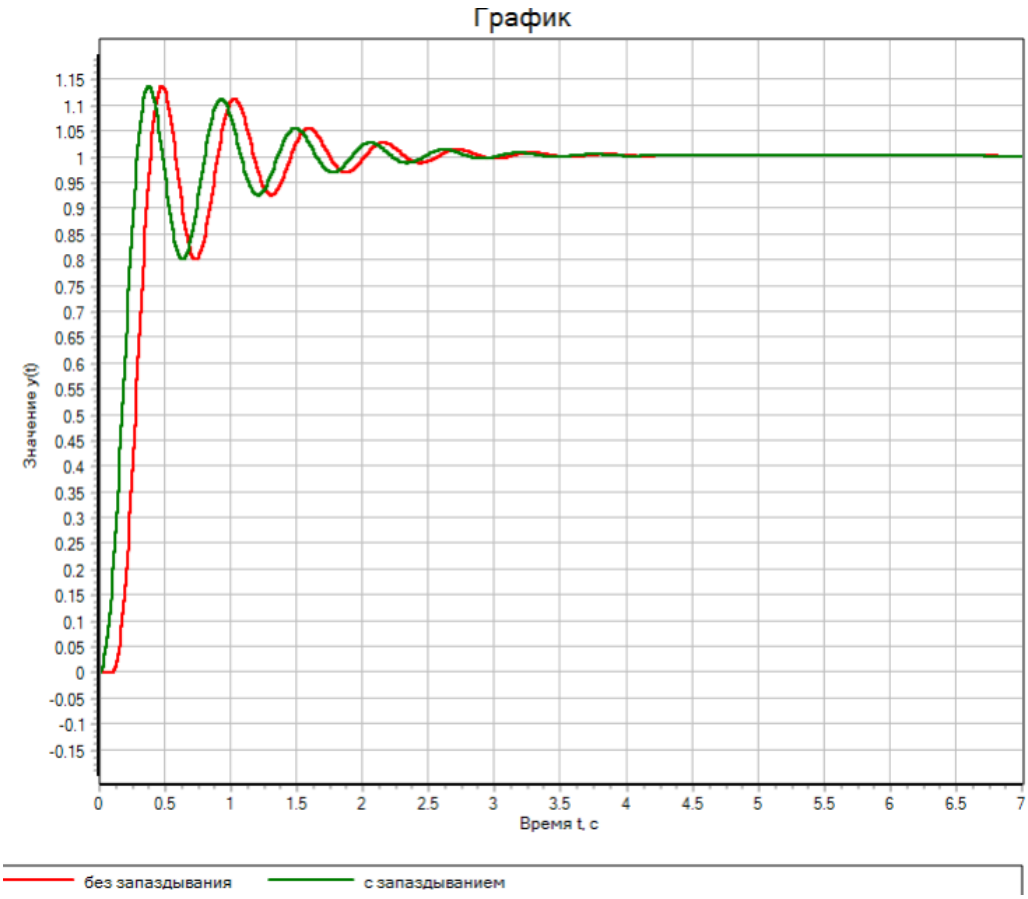

График 9 – Переходный процесс в SimInTech

Критерий улучшился в $\frac{1{,}1663}{0{,}24811} = 4,7010$ раза.

Расчёт функции чувствительности

При реализации алгоритма параметрической оптимизации градиентным методом необходимы значения каждой составляющей вектор-градиента целевой функции

$$\frac{\partial I(x(t,q))}{\partial q_j}$$

В общем виде составляющие градиента имеют вид:

$$\frac{\partial I}{\partial q_j} = \int_0^\infty \frac{\partial F(x(t,q),t)}{\partial x} \cdot \frac{\partial x(t,q)}{\partial q_j} dt$$

Учитывая, что для исследуемой системы ошибка определяется как $x(t,q) = g(t) - y(t,q)$, а входной сигнал не зависит от параметров регулятора, получим:

$$\frac{\partial x(t,q)}{\partial q_j} = \frac{\partial (g(t) - y(t,q))}{\partial q_j} = -\frac{\partial y(t,q)}{\partial q_j} = -\xi_{j(t)}$$

Здесь $\xi_j = \frac{\partial y(t,q)}{\partial q_j}$ — называется функцией чувствительности, которая характеризует влияние j-го настраиваемого параметра q_j на выходную координату объекта регулирования.

Таким образом, для вычисления градиента необходимо предварительно вычислить три функции чувствительности ξ_j, так как ПИД-регулятор имеет три настраиваемых параметра ($j = 1, 2, 3$).

В соответствии со структурной схемой исследуемой системы:

$$y(t,q) = W_{\text{об}(p)} u(t,q)$$

$$u(t,q) = W_{\text{рег}(p)} x(t) = q_1 x(t) + \frac{q_2}{p} x(t) + q_3 p x(t)$$

Дифференцируя частным образом правую и левую части и учитывая независимость $W_{\text{об}(p)}$ от настраиваемых параметров q_j, получим:

$$\xi_j = \frac{\partial y(t,q)}{\partial q_j} = W_{\text{об}(p)} \frac{\partial u(t,q)}{\partial q_j}, \quad j = 1, 2, 3$$

Т.е. функция чувствительности представляет собой выходную координату модели объекта регулирования с входной координатой $\frac{\partial u(t,q)}{\partial q_j}$.

Таким образом, для вычисления значений функций чувствительности ξ_j необходимо подать на вход объекта регулирования сигнал, соответствующий j-ой частной производной управляющего сигнала $\frac{\partial u(t,q)}{\partial q_j}$.

Для ПИД-регулятора $u(t,q) = \left(q_1 + \frac{q_2}{p} + q_3 p\right) x(t)$ частные производные управляющего сигнала имеют вид:

$$\begin{cases} \frac{\partial u}{\partial q_1} = x(t) - q_1 \xi_1(t) - \frac{q_2}{p} \xi_1(t) - q_3 p \xi_1(t) \\ \frac{\partial u}{\partial q_2} = \frac{1}{p} x(t) - q_1 \xi_2(t) - \frac{q_2}{p} \xi_2(t) - q_3 p \xi_2(t) \\ \frac{\partial u}{\partial q_3} = p x(t) - q_1 \xi_3(t) - \frac{q_2}{p} \xi_3(t) - q_3 p \xi_3(t) \end{cases}$$

Полученные сигналы $\frac{\partial u}{\partial q_j}$ подаются на вход модели объекта регулирования $W_{\text{об}(p)}$, с выхода которой снимаются значения функций чувствительности $\xi_1(t), \xi_2(t), \xi_3(t)$.

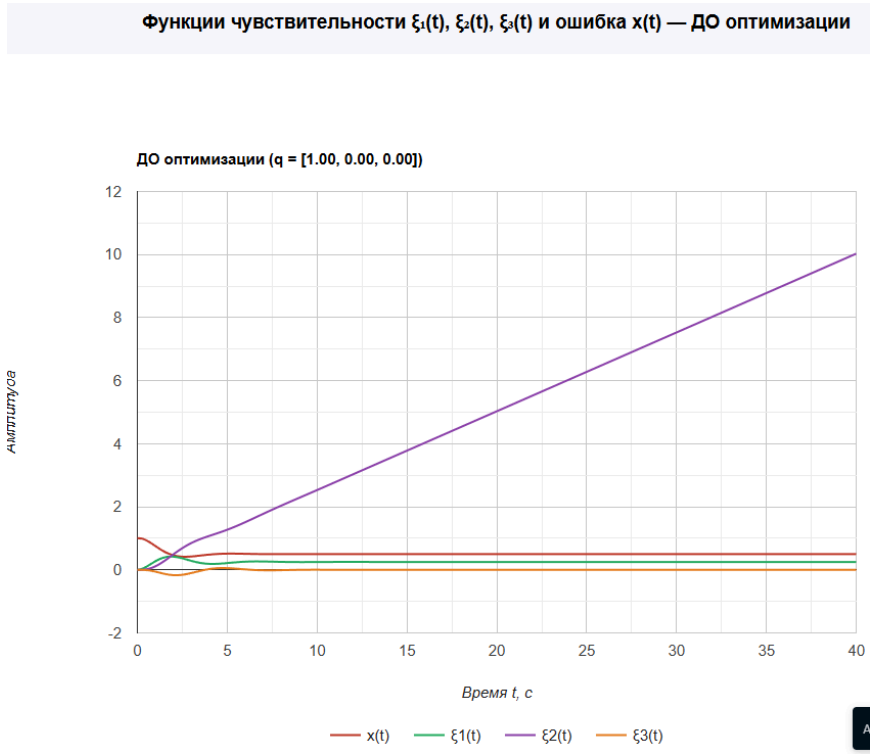

График 10 – Функции чувствительности $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$ и ошибка x(t) ДО оптимизации

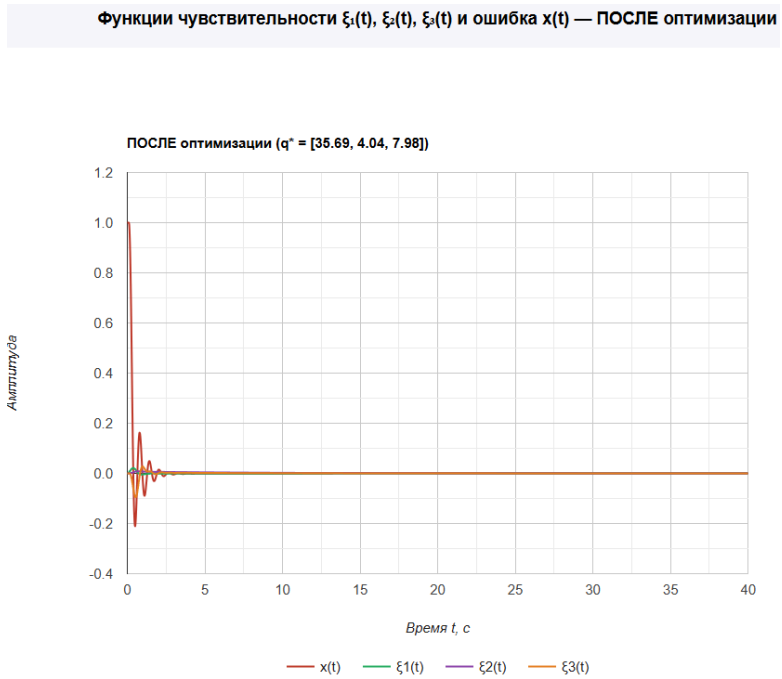

График 11 – Функции чувствительности $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$ и ошибка x(t) ПОСЛЕ оптимизации

Алгоритм градиентного спуска с нормализацией и адаптивным шагом

Основная идея метода: Алгоритм ищет минимум, двигаясь строго против градиента. Чтобы избежать резких скачков параметров на крутых участках, вектор градиента нормализуется — так мы получаем только направление, а длину шага h контролируем сами.

Алгоритм:

1) Задаются начальные значения параметров регулятора $q^0 = [q_1^0, q_2^0, q_3^0]$, начальный шаг $h^0 > 0$, точность по градиенту $\varepsilon > 0$ и минимально допустимый шаг $h_{\min}$. Полагаем номер итерации $l = 0$.

2) Для текущих параметров q^l численно решается система дифференциальных уравнений. Определяются:
   - ошибка регулирования $x(t, q^l)$;
   - функции чувствительности $\xi_j(t, q^l)$ (с учетом запаздывания τ);
   - текущее значение критерия качества $I(q^l) = \int_0^{T_{\text{мод}}} x^2(t, q^l)dt$.

3) Вычисляются компоненты градиента:
$$\frac{\partial I}{\partial q_j} = -2\int_0^{T_{\text{мод}}} x(t, q^l) \cdot \xi_j(t, q^l)dt, \quad j = 1, 2, 3$$
   Вычисляется евклидова норма градиента:
$$\|\nabla I\| = \sqrt{\sum_{j=1}^{3}\left(\frac{\partial I}{\partial q_j}\right)^2}$$
   Если $\|\nabla I\| < \varepsilon$, алгоритм завершается.

4) Вычисляются новые значения параметров («пробная точка»):
$$q_j^{try} = q_j^l - h^l \cdot \frac{\left(\partial \frac{I}{\partial} q_j\right)}{\|\nabla I\|}, \quad j = 1, 2, 3$$
   Применяется ограничение $q_j^{try} = \max\left(0, q_j^{try}\right)$, чтобы исключить отрицательные значения коэффициентов.

5) Проводится моделирование системы с параметрами q^{try} и вычисляется новый критерий $I(q^{try})$.
   - **Если** $I(q^{try}) < I(q^l)$ (Успех): Принимаем новые параметры: $q^{l+1} = q^{try}$. Увеличиваем шаг для следующей итерации: $h^{l+1} = h^l \cdot 1.05$. Переходим к Шагу 2 с $l = l + 1$.
   - **Если** $I(q^{try}) \geq I(q^l)$ (Неудача): Параметры не обновляются: $q^{l+1} = q^l$. Уменьшаем шаг: $h^l = h^l \cdot 0.5$. Если $h^l < h_{\min}$, алгоритм завершается. В противном случае возвращаемся к Шагу 4 для повторного совершения пробного шага из той же точки.

Результаты моделирования:

Оптимальные параметры: q* = [35.6860, 4.0382, 7.9786]
Критерий: I* = 0.2503
Градиент: [-0.000330, -0.000029, -0.000031]
Норма градиента: ||∇I|| = 0.000332

График 12 – Оптимальные параметры ПИД-регулятора, полученные методом градиентного спуска с нормализацией

График 13 – Подынтегральные выражения градиента

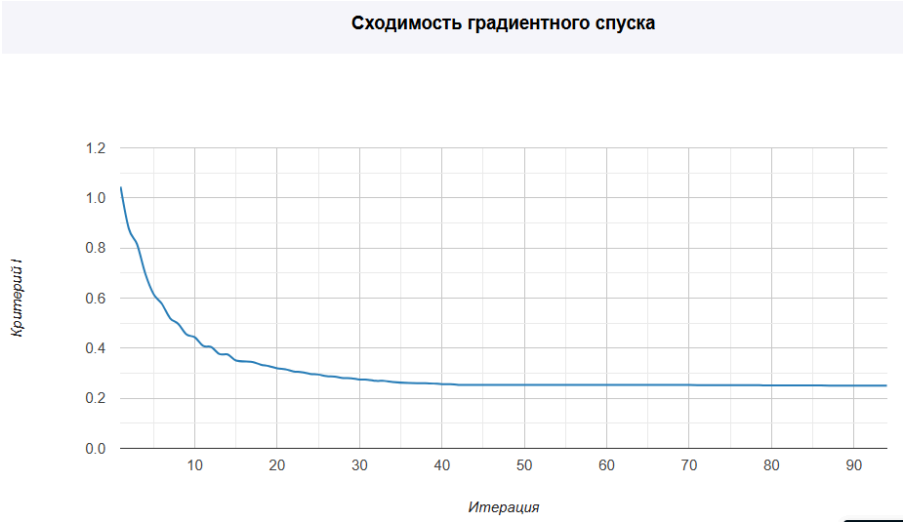

График 14 – Сходимость градиентного спуска

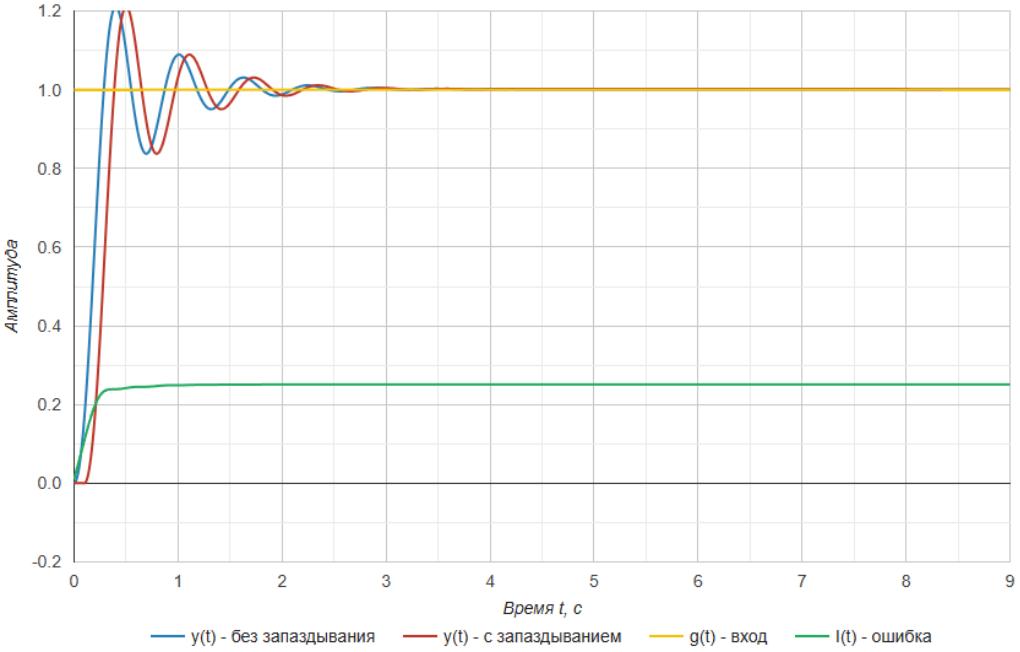

График 15 – Переходный процесс с оптимальными параметрами (q_1 = 35.686, q_2 = 4.0382, q_3 = 7.9786)

Алгоритм наискорейшего спуска с линейным поиском ($I_3(2)$)

Основная идея метода: Метод наискорейшего спуска — это градиентный метод оптимизации, в котором на каждой итерации движение осуществляется в направлении антиградиента (направлении наибыстрейшего убывания функционала). Ключевое отличие от простого градиентного спуска — наличие внутреннего цикла линейного поиска: шаг в выбранном направлении выполняется до тех пор, пока функционал уменьшается.

Критерий оптимизации ($I_3(2)$): В отличие от стандартного критерия $I_3 = \int x^2(t)dt$, используется модифицированный функционал, минимизирующий отклонение от эталонной модели:

$$I_3^{\{(2)\}} = \int_0^L \left(y(t) - y_{\text{эт}(t)}\right)^2 dt$$

где $y(t)$ — выходной сигнал оптимизируемой системы, $y_{\text{эт}(t)}$ — выход эталонной модели.

Эталонная модель: Инерционное звено первого порядка:

$$W_{\text{эт}(p)} = \frac{k_{\text{эт}}}{T_{\text{эт}}p + 1}$$

с параметрами $k_{\text{эт}} = 1$, $T_{\text{эт}} = 0.4$.

Расчёт выхода эталонной модели производится численно методом Рунге-Кутты 4-го порядка:

$$\begin{cases} k_1 = \text{dt}\left(-\frac{y_{\text{эт}}}{T_{\text{эт}}} + \frac{k_{\text{эт}}}{T_{\text{эт}}} * g\right) \\ k_2 = \text{dt}\left(-\frac{y_{\text{эт}}+\frac{k_1}{2}}{T_{\text{эт}}} + \frac{k_{\text{эт}}}{T_{\text{эт}}} * g\right) \\ k_3 = \text{dt}\left(-\frac{y_{\text{эт}}+\frac{k_2}{2}}{T_{\text{эт}}} + \frac{k_{\text{эт}}}{T_{\text{эт}}} * g\right) \\ k_4 = \text{dt}\left(-\frac{y_{\text{эт}}+k_3}{T_{\text{эт}}} + \frac{k_{\text{эт}}}{T_{\text{эт}}} * g\right) \end{cases}$$

$$y_{\text{эт}} = y_{\text{эт}} + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Градиент функционала: Для критерия $I_3(2)$ градиент вычисляется по формуле:

$$\frac{\partial I}{\partial q_j} = 2\int_0^L \left(y(t) - y_{\text{эт}(t)}\right) \cdot \xi_{j(t)} dt, \quad j = 1, 2, 3$$

где $\xi_{j(t)} = \frac{\partial y(t)}{\partial q_j}$ — функции чувствительности.

Алгоритм:

1) Задаются начальные значения параметров регулятора $q^0 = [q_1^0, q_2^0, q_3^0]$, начальный шаг $h^0 > 0$, точность по градиенту $\varepsilon = 10^{-5}$. Полагаем номер итерации $k = 0$.

2) Для текущих параметров q^k численно решается система дифференциальных уравнений. Определяются:
   - ошибка регулирования $x(t, q^k) = g(t) - y(t, q^k)$;

- выход эталонной модели $y_{\text{эт}(t)}$;
- разность $\Delta(t) = y(t, q^k) - y_{\text{эт}(t)}$;
- функции чувствительности $\xi_j(t, q^k)$ (с учетом запаздывания τ);
- текущее значение критерия качества $I(q^k) = \int_0^L \Delta^2(t)dt$.

3) Вычисляются компоненты градиента:

$$\frac{\partial I}{\partial q_j} = 2\int_0^L \Delta(t) \cdot \xi_j(t, q^k)dt, \quad j = 1, 2, 3$$

Вычисляется евклидова норма градиента:

$$\|\nabla I\| = \sqrt{\sum_{j=1}^{3} \left(\frac{\partial I}{\partial q_j}\right)^2}$$

Если $\|\nabla I\| < \varepsilon$, алгоритм завершается.

4) Определяется направление наискорейшего спуска (единичный вектор):

$$S = -\frac{\nabla I}{\|\nabla I\|}, \quad \text{т.е.} \quad S_j = -\frac{\left(\frac{\partial I}{\partial q_j}\right)}{\|\nabla I\|}, \quad j = 1, 2, 3$$

5) Линейный поиск вдоль направления S:
   а) Полагаем $q_{\text{лучш}} = q^k$, $I_{\text{лучш}} = I(q^k)$, $h_{\text{тек}} = h^k$, счётчик шагов $n = 0$.
   б) В цикле (максимум 100 шагов):
      1) Вычислить пробные параметры: $q_{\text{проб}} = q^k + h_{\text{тек}} \cdot S$.
      2) Применить ограничение: $q_{\text{проб}} = \max(0, q_{\text{проб}})$.
      3) Вычислить $I(q_{\text{проб}})$.
      4) Если $I(q_{\text{проб}}) < I_{\text{лучш}}$:
         а) Принять $q_{\text{лучш}} = q_{\text{проб}}$, $I_{\text{лучш}} = I(q_{\text{проб}})$.
         б) Увеличить шаг: $h_{\text{тек}} = h_{\text{тек}} \cdot 1.2$.
         в) Увеличить счётчик: $n = n + 1$.
         г) Продолжить цикл.
      5) Иначе (перелёт через минимум): выйти из цикла.

6) Обновить параметры: $q^{k+1} = q_{\text{лучш}}$, $I(q^{k+1}) = I_{\text{лучш}}$.

7) Если на шаге 5 не было сделано ни одного успешного шага ($n = 0$):
   а) Уменьшить начальный шаг: $h^k = \frac{h^k}{2}$.
   б) Если $h^k < 10^{-7}$, завершить алгоритм.
   Иначе сбросить шаг к начальному: $h^{k+1} = h^0$.

8) Увеличить номер итерации: $k = k + 1$. Вернуться к шагу 2.

Код реализации:

```
<!doctype html>
<html>
 <head>
  <meta charset="utf-8" />
  <script
   type="text/javascript"
```

```
  src="https://www.gstatic.com/charts/loader.js"
></script>
<style>
  body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 20px;
    background: #f5f6fa;
  }
  .container {
    display: flex;
    justify-content: center;
    margin: 20px 0;
  }
  #result_info {
    background: #fff;
    padding: 15px;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
    max-width: 800px;
    margin: 20px auto;
    text-align: center;
    font-size: 16px;
  }
</style>
<script type="text/javascript">
  google.charts.load("current", { packages: ["corechart"] });
  google.charts.setOnLoadCallback(drawChart);

  // Функция симуляции - считает I и градиент
  function simulate(q1, q2, q3) {
    var T = 1,
      k = 1,
      zeta = 0.75,
      tau = 0.1,
      g = 1,
      dt = 0.01,
      L = 40;
    var z1 = 0,
      z2 = 0;
    var x,
      x_prev = g,
      intx = 0,
      dx,
      u;
    var I = 0;
    var eta1 = 0,
      eta2 = 0,
      eta3 = 0;
    var nu1 = 0,
      nu2 = 0,
      nu3 = 0;
    var int_xi1 = 0,
      int_xi2 = 0,
      int_xi3 = 0;
```

```
var gradI1 = 0,
  gradI2 = 0,
  gradI3 = 0;
var data = [],
  gradData = [];
var t_ = 0;

// Параметры эталонной модели
var T_ref = 0.6,
  k_ref = 1;
var y_ref_prev = 0;

// Запаздывание
var delaySteps = Math.max(1, Math.round(tau / dt));
var historyY = new Array(delaySteps).fill(0);
var historyXi1 = new Array(delaySteps).fill(0);
var historyXi2 = new Array(delaySteps).fill(0);
var historyXi3 = new Array(delaySteps).fill(0);

while (t_ <= L) {
  var y_now = z1;

  // Достаём задержанные значения
  var y_delayed = historyY.shift();
  var xi1_delayed = historyXi1.shift();
  var xi2_delayed = historyXi2.shift();
  var xi3_delayed = historyXi3.shift();

  // Ошибка по задержанному сигналу
  x = g - y_delayed;
  intx += x * dt;
  dx = (x - x_prev) / dt;
  x_prev = x;
  u = q1 * x + q2 * intx + q3 * dx;

  // Эталонная модель (Рунге-Кутта 4)
  var obrat_g = g;
  var k1_ref = dt * (-(y_ref_prev / T_ref) + (k_ref / T_ref) * obrat_g);
  var k2_ref =
    dt *
    (-(y_ref_prev + k1_ref / 2) / T_ref + (k_ref / T_ref) * obrat_g);
  var k3_ref =
    dt *
    (-(y_ref_prev + k2_ref / 2) / T_ref + (k_ref / T_ref) * obrat_g);
  var k4_ref =
    dt * (-(y_ref_prev + k3_ref) / T_ref + (k_ref / T_ref) * obrat_g);
  var y_ref =
    y_ref_prev + (1 / 6) * (k1_ref + 2 * k2_ref + 2 * k3_ref + k4_ref);
  y_ref_prev = y_ref;

  // Разница с эталоном
  var diff = y_delayed - y_ref;

  // Функционал I
  I += diff * diff * dt;
```

```
// Обновляем историю
historyY.push(y_now);
historyXi1.push(eta1);
historyXi2.push(eta2);
historyXi3.push(eta3);

// Производные управления
// d(dx)/dqj теперь тоже зависит от изменения задержанной xi
var dxi1_dt = nu1,
  dxi2_dt = nu2,
  dxi3_dt = nu3;

var du_dq1 = x - q1 * xi1_delayed - q2 * int_xi1 - q3 * dxi1_dt;
var du_dq2 = intx - q1 * xi2_delayed - q2 * int_xi2 - q3 * dxi2_dt;
var du_dq3 = dx - q1 * xi3_delayed - q2 * int_xi3 - q3 * dxi3_dt;

// Уравнения чувствительности (динамика объекта)
var dnu1_dt =
  (k / (T * T)) * du_dq1 -
  ((2 * zeta) / T) * nu1 -
  (1 / (T * T)) * eta1;
var dnu2_dt =
  (k / (T * T)) * du_dq2 -
  ((2 * zeta) / T) * nu2 -
  (1 / (T * T)) * eta2;
var dnu3_dt =
  (k / (T * T)) * du_dq3 -
  ((2 * zeta) / T) * nu3 -
  (1 / (T * T)) * eta3;

// Эйлер для чувствительности
eta1 += nu1 * dt;
eta2 += nu2 * dt;
eta3 += nu3 * dt;
nu1 += dnu1_dt * dt;
nu2 += dnu2_dt * dt;
nu3 += dnu3_dt * dt;

int_xi1 += xi1_delayed * dt;
int_xi2 += xi2_delayed * dt;
int_xi3 += xi3_delayed * dt;

// Новый градиент: ∂I/∂qj = 2 ∫ (y - y_эт) · ξj dt
gradI1 += 2 * diff * xi1_delayed * dt;
gradI2 += 2 * diff * xi2_delayed * dt;
gradI3 += 2 * diff * xi3_delayed * dt;

data.push([t_, x, eta1, eta2, eta3, y_ref, y_delayed]);
gradData.push([
  t_,
  2 * diff * xi1_delayed,
  2 * diff * xi2_delayed,
  2 * diff * xi3_delayed,
]);
```

```
    // Рунге-Кутта 4 для самого объекта
    var k1 = dt * (z2 - ((2 * zeta) / T) * y_now);
    var m1 = dt * ((k / (T * T)) * u - y_now / (T * T));
    var k2 = dt * (z2 + m1 / 2 - ((2 * zeta) / T) * (y_now + k1 / 2));
    var m2 = dt * ((k / (T * T)) * u - (1 / (T * T)) * (y_now + k1 / 2));
    var k3 = dt * (z2 + m2 / 2 - ((2 * zeta) / T) * (y_now + k2 / 2));
    var m3 = dt * ((k / (T * T)) * u - (1 / (T * T)) * (y_now + k2 / 2));
    var k4 = dt * (z2 + m3 - ((2 * zeta) / T) * (y_now + k3));
    var m4 = dt * ((k / (T * T)) * u - (1 / (T * T)) * (y_now + k3));

    z1 += (1 / 6) * (k1 + 2 * k2 + 2 * k3 + k4);
    z2 += (1 / 6) * (m1 + 2 * m2 + 2 * m3 + m4);
    t_ += dt;
  }
  return {
    I: I,
    grad: [gradI1, gradI2, gradI3],
    data: data,
    gradData: gradData,
  };
}

// Метод наискорейшего спуска (градиент + линейный поиск)
function gradientNorm(grad) {
  return Math.sqrt(
    grad[0] * grad[0] + grad[1] * grad[1] + grad[2] * grad[2],
  );
}

function clampParams(q) {
  return q.map((val) => Math.max(0.0, val));
}

function runSteepestDescent(qStart, hInit, maxIter) {
  var q = qStart.slice();
  var h = hInit;
  var history = [];
  var totalIter = 0;

  // Начальное состояние
  var currentResult = simulate(q[0], q[1], q[2]);
  var currentI = currentResult.I;

  while (totalIter < maxIter) {
    var grad = currentResult.grad;
    var norm = gradientNorm(grad);

    // Проверка сходимости
    if (norm < 1e-5) break;

    // Направление спуска
    var S = [-grad[0] / norm, -grad[1] / norm, -grad[2] / norm];

    // Линейный поиск
```

```
  var qBest = q.slice();
  var IBest = currentI;
  var stepCount = 0;
  var hCurrent = h;

  while (stepCount < 100) {
    var qNext = [];
    for (var i = 0; i < 3; i++) {
      qNext.push(q[i] + hCurrent * S[i]);
    }
    qNext = clampParams(qNext);

    var nextResult = simulate(qNext[0], qNext[1], qNext[2]);
    var nextI = nextResult.I;

    if (nextI < IBest) {
      qBest = qNext.slice();
      IBest = nextI;
      q = qNext.slice();
      hCurrent *= 1.2;
      stepCount++;
    } else {
      break;
    }
  }

  q = qBest.slice();
  currentI = IBest;
  currentResult = simulate(q[0], q[1], q[2]);

  // Записываем историю
  history.push({
    iter: totalIter + 1,
    q1: q[0],
    q2: q[1],
    q3: q[2],
    I: currentI,
    norm: norm,
  });

  totalIter++;

  if (stepCount === 0) {
    h *= 0.5;
    if (h < 1e-7) break;
  } else {
    h = hInit;
  }
}

return {
  q: q,
  I: currentI,
  grad: currentResult.grad,
  history: history,
```

```
  };
}

function drawChart() {
  // 3 набора параметров
  var qInit_sets = [
    [1.0, 0.0, 0.0],
    [0.1, 0.01, 7.0],
    [10.0, 2.0, 0.0],
  ];

  var qInit = qInit_sets[0];
  var hStart = 0.5;
  var maxIter = 1000;
  var optResult = runSteepestDescent(qInit, hStart, maxIter);

  // Запускаем оптимизацию для всех 3 наборов
  var optResult1 = runSteepestDescent(qInit_sets[0], hStart, maxIter);
  var optResult2 = runSteepestDescent(qInit_sets[1], hStart, maxIter);
  var optResult3 = runSteepestDescent(qInit_sets[2], hStart, maxIter);

  // Вывод оптимальных параметров
  document.getElementById("result_info").innerHTML =
    "<strong>Оптимальные параметры:</strong> q* = [" +
    optResult.q[0].toFixed(4) +
    ", " +
    optResult.q[1].toFixed(4) +
    ", " +
    optResult.q[2].toFixed(4) +
    "]<br>" +
    "<strong>Критерий:</strong> I* = " +
    optResult.I.toFixed(4) +
    "<br>" +
    "<strong>Градиент:</strong> [" +
    optResult.grad[0].toFixed(6) +
    ", " +
    optResult.grad[1].toFixed(6) +
    ", " +
    optResult.grad[2].toFixed(6) +
    "]<br>" +
    "<strong>Норма градиента:</strong> ||∇I|| = " +
    gradientNorm(optResult.grad).toFixed(6);

  var simBefore = simulate(qInit[0], qInit[1], qInit[2]);
  var simAfter = simulate(optResult.q[0], optResult.q[1], optResult.q[2]);

  // === ГРАФИКИ ДЛЯ ТРЁХ НАБОРОВ НАЧАЛЬНЫХ ПАРАМЕТРОВ ===
  // Общий график: все 3 набора ДО оптимизации + ПОСЛЕ оптимизации +
эталонная модель

  var simSet1_before = simulate(
    qInit_sets[0][0],
    qInit_sets[0][1],
    qInit_sets[0][2],
  );
```

```
var simSet2_before = simulate(
  qInit_sets[1][0],
  qInit_sets[1][1],
  qInit_sets[1][2],
);
var simSet3_before = simulate(
  qInit_sets[2][0],
  qInit_sets[2][1],
  qInit_sets[2][2],
);

var simSet1_after = simulate(
  optResult1.q[0],
  optResult1.q[1],
  optResult1.q[2],
);
var simSet2_after = simulate(
  optResult2.q[0],
  optResult2.q[1],
  optResult2.q[2],
);
var simSet3_after = simulate(
  optResult3.q[0],
  optResult3.q[1],
  optResult3.q[2],
);

// График 1: Переходные процессы (3 набора ДО и ПОСЛЕ + эталон)
var transAllData = [
  [
    "t",
    "Набор 1: q=[1.0, 0.0, 0.0]",
    "Набор 2: q=[0.1, 0.01, 7.0]",
    "Набор 3: q=[10.0, 2.0, 0.0]",
    "q*=[" +
      optResult1.q[0].toFixed(2) +
      ", " +
      optResult1.q[1].toFixed(2) +
      ", " +
      optResult1.q[2].toFixed(2) +
      "]",
    "y_эт(t)",
  ],
];
for (var i = 0; i < simSet1_before.data.length; i++) {
  transAllData.push([
    simSet1_before.data[i][0],
    simSet1_before.data[i][6],
    simSet2_before.data[i][6],
    simSet3_before.data[i][6],
    simSet1_after.data[i][6],
    simSet1_after.data[i][5],
  ]);
}
var dataTransAll = google.visualization.arrayToDataTable(transAllData);
```

```
new google.visualization.LineChart(
  document.getElementById("all_sets_chart"),
).draw(dataTransAll, {
  curveType: "function",
  hAxis: { title: "Время t, с" },
  vAxis: { title: "Амплитуда" },
  legend: { position: "bottom" },
  colors: ["#e74c3c", "#3498db", "#f1c40f", "#27ae60", "#2ecc71"],
  lineWidth: 2,
  title: "Переходные процессы: 3 набора",
});

// График 2: Подынтегральные выражения градиента
var gradDataAll = [
  ["t", "d(∂I/∂q₁)/dt", "d(∂I/∂q₂)/dt", "d(∂I/∂q₃)/dt"],
];
for (var i = 0; i < simSet1_after.gradData.length; i++)
  gradDataAll.push(simSet1_after.gradData[i]);
var dataGradAll = google.visualization.arrayToDataTable(gradDataAll);
new google.visualization.LineChart(
  document.getElementById("grad_all_chart"),
).draw(dataGradAll, {
  curveType: "function",
  hAxis: { title: "Время t, с" },
  vAxis: { title: "Амплитуда" },
  legend: { position: "bottom" },
  colors: ["#c0392b", "#27ae60", "#8e44ad"],
  lineWidth: 2,
  title: "Подынтегральные выражения градиента",
});

// График 3: Функции чувствительности
var sensDataAll = [["t", "ξ1(t)", "ξ2(t)", "ξ3(t)"]];
for (var i = 0; i < simSet1_after.data.length; i++) {
  sensDataAll.push([
    simSet1_after.data[i][0],
    simSet1_after.data[i][2],
    simSet1_after.data[i][3],
    simSet1_after.data[i][4],
  ]);
}
var dataSensAll = google.visualization.arrayToDataTable(sensDataAll);
new google.visualization.LineChart(
  document.getElementById("sens_all_chart"),
).draw(dataSensAll, {
  curveType: "function",
  hAxis: { title: "Время t, с" },
  vAxis: { title: "Амплитуда" },
  legend: { position: "bottom" },
  colors: ["#27ae60", "#8e44ad", "#e67e22"],
  lineWidth: 2,
  title: "Функции чувствительности",
});

// График 4: Сходимость I
```

```
var convAllData = [["Итерация", "Набор 1", "Набор 2", "Набор 3"]];
var maxLen = Math.min(
  100,
  optResult1.history.length,
  optResult2.history.length,
  optResult3.history.length,
);
for (var i = 0; i < maxLen; i++) {
  var v1 = optResult1.history[i].I;
  var v2 = optResult2.history[i].I;
  var v3 = optResult3.history[i].I;
  convAllData.push([i + 1, v1, v2, v3]);
}
var dataConvAll = google.visualization.arrayToDataTable(convAllData);
new google.visualization.LineChart(
  document.getElementById("conv_all_chart"),
).draw(dataConvAll, {
  curveType: "function",
  hAxis: { title: "Итерация" },
  vAxis: { title: "Критерий I" },
  legend: { position: "bottom" },
  colors: ["#e74c3c", "#3498db", "#f1c40f"],
  lineWidth: 2,
  title: "Сходимость I (I* = " + optResult1.I.toFixed(4) + ")",
});

// Сходимость q1
var F_q1 = [["Итерация", "Набор 1", "Набор 2", "Набор 3"]];
var maxLenQ = Math.max(
  optResult1.history.length,
  optResult2.history.length,
  optResult3.history.length,
);
for (var i = 0; i < maxLenQ; i++) {
  var v1 =
    i < optResult1.history.length
      ? optResult1.history[i].q1
      : optResult1.q[0];
  var v2 =
    i < optResult2.history.length
      ? optResult2.history[i].q1
      : optResult2.q[0];
  var v3 =
    i < optResult3.history.length
      ? optResult3.history[i].q1
      : optResult3.q[0];
  F_q1.push([i + 1, v1, v2, v3]);
}
var dataQ1 = google.visualization.arrayToDataTable(F_q1);
new google.visualization.LineChart(
  document.getElementById("q1_conv_chart"),
).draw(dataQ1, {
  curveType: "function",
  hAxis: { title: "Итерация" },
  vAxis: { title: "q1" },
```

```
  legend: { position: "bottom" },
  colors: ["#e74c3c", "#3498db", "#f1c40f"],
  lineWidth: 2,
  title: "Сходимость q1 → " + optResult1.q[0].toFixed(2),
});

// Сходимость q2
var F_q2 = [["Итерация", "Набор 1", "Набор 2", "Набор 3"]];
for (var i = 0; i < maxLenQ; i++) {
  var v1 =
    i < optResult1.history.length
      ? optResult1.history[i].q2
      : optResult1.q[1];
  var v2 =
    i < optResult2.history.length
      ? optResult2.history[i].q2
      : optResult2.q[1];
  var v3 =
    i < optResult3.history.length
      ? optResult3.history[i].q2
      : optResult3.q[1];
  F_q2.push([i + 1, v1, v2, v3]);
}
var dataQ2 = google.visualization.arrayToDataTable(F_q2);
new google.visualization.LineChart(
  document.getElementById("q2_conv_chart"),
).draw(dataQ2, {
  curveType: "function",
  hAxis: { title: "Итерация" },
  vAxis: { title: "q2" },
  legend: { position: "bottom" },
  colors: ["#e74c3c", "#3498db", "#f1c40f"],
  lineWidth: 2,
  title: "Сходимость q2 → " + optResult1.q[1].toFixed(2),
});

// Сходимость q3
var F_q3 = [["Итерация", "Набор 1", "Набор 2", "Набор 3"]];
for (var i = 0; i < maxLenQ; i++) {
  var v1 =
    i < optResult1.history.length
      ? optResult1.history[i].q3
      : optResult1.q[2];
  var v2 =
    i < optResult2.history.length
      ? optResult2.history[i].q3
      : optResult2.q[2];
  var v3 =
    i < optResult3.history.length
      ? optResult3.history[i].q3
      : optResult3.q[2];
  F_q3.push([i + 1, v1, v2, v3]);
}
var dataQ3 = google.visualization.arrayToDataTable(F_q3);
new google.visualization.LineChart(
```

```
      document.getElementById("q3_conv_chart"),
    ).draw(dataQ3, {
      curveType: "function",
      hAxis: { title: "Итерация" },
      vAxis: { title: "q3" },
      legend: { position: "bottom" },
      colors: ["#e74c3c", "#3498db", "#f1c40f"],
      lineWidth: 2,
      title: "Сходимость q3 → " + optResult1.q[2].toFixed(2),
    });
  }
 </script>
</head>
<body>
 <div id="result_info"></div>

 <h2 style="text-align: center; color: #2c3e50">
  Пункт 3: Подтверждение работоспособности алгоритма оптимизации
 </h2>

 <h3 style="text-align: center">3 набора</h3>
 <div class="container">
  <div id="all_sets_chart" style="width: 1200px; height: 700px"></div>
 </div>

 <h3 style="text-align: center">Подынтегральные выражения градиента</h3>
 <div class="container">
  <div id="grad_all_chart" style="width: 1200px; height: 500px"></div>
 </div>

 <h3 style="text-align: center">Функции чувствительности</h3>
 <div class="container">
  <div id="sens_all_chart" style="width: 1200px; height: 500px"></div>
 </div>

 <h3 style="text-align: center">Сходимость I</h3>
 <div class="container">
  <div id="conv_all_chart" style="width: 1200px; height: 500px"></div>
 </div>

 <h2 style="text-align: center; color: #2c3e50">
  Пункт 3.2: Сходимость параметров ПИД-регулятора
 </h2>

 <h3 style="text-align: center">Сходимость q1</h3>
 <div class="container">
  <div id="q1_conv_chart" style="width: 1200px; height: 500px"></div>
 </div>

 <h3 style="text-align: center">Сходимость q2</h3>
 <div class="container">
  <div id="q2_conv_chart" style="width: 1200px; height: 500px"></div>
 </div>

 <h3 style="text-align: center">Сходимость q3</h3>
```

```
    <div class="container">
      <div id="q3_conv_chart" style="width: 1200px; height: 500px"></div>
    </div>

    <h3 style="text-align: center">Сходимость составляющих градиента (Набор 1)</h3>
    <div class="container">
      <div id="grad_conv_chart" style="width: 1200px; height: 500px"></div>
    </div>

    <h3 style="text-align: center">Сходимость составляющих градиента (Набор 2)</h3>
    <div class="container">
      <div id="grad_conv2_chart" style="width: 1200px; height: 500px"></div>
    </div>

    <h3 style="text-align: center">Сходимость составляющих градиента (Набор 3)</h3>
    <div class="container">
      <div id="grad_conv3_chart" style="width: 1200px; height: 500px"></div>
    </div>
  </body>
</html>
```

Результаты моделирования:

Оптимальные параметры: q* = [19.1530, 1.5419, 10.4933]
Критерий: I* = 0.0087

График 16 – Оптимальные параметры ПИД-регулятора, полученные методом наискорейшего спуска

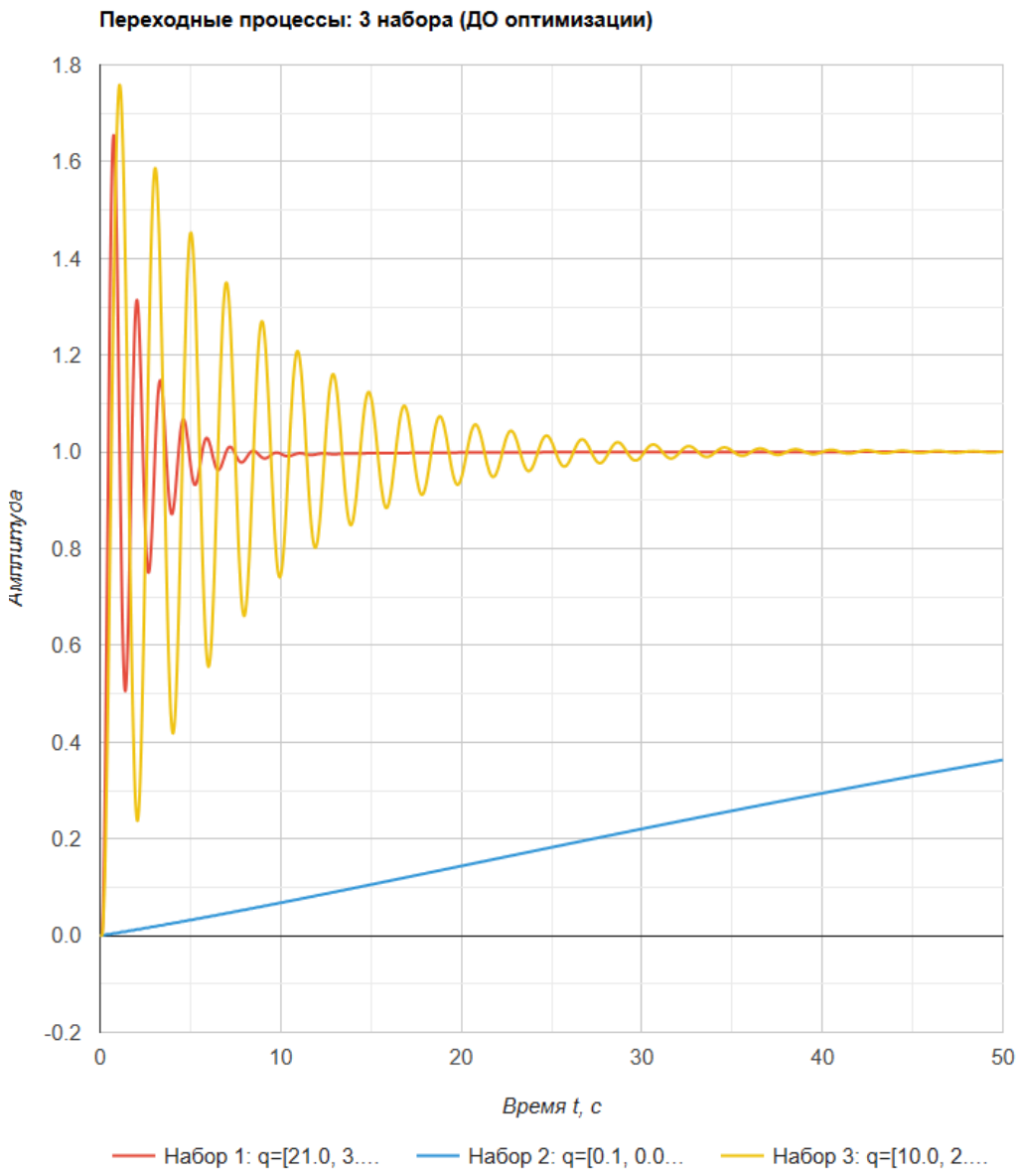

График 17 – Переходные процессы: 3 набора

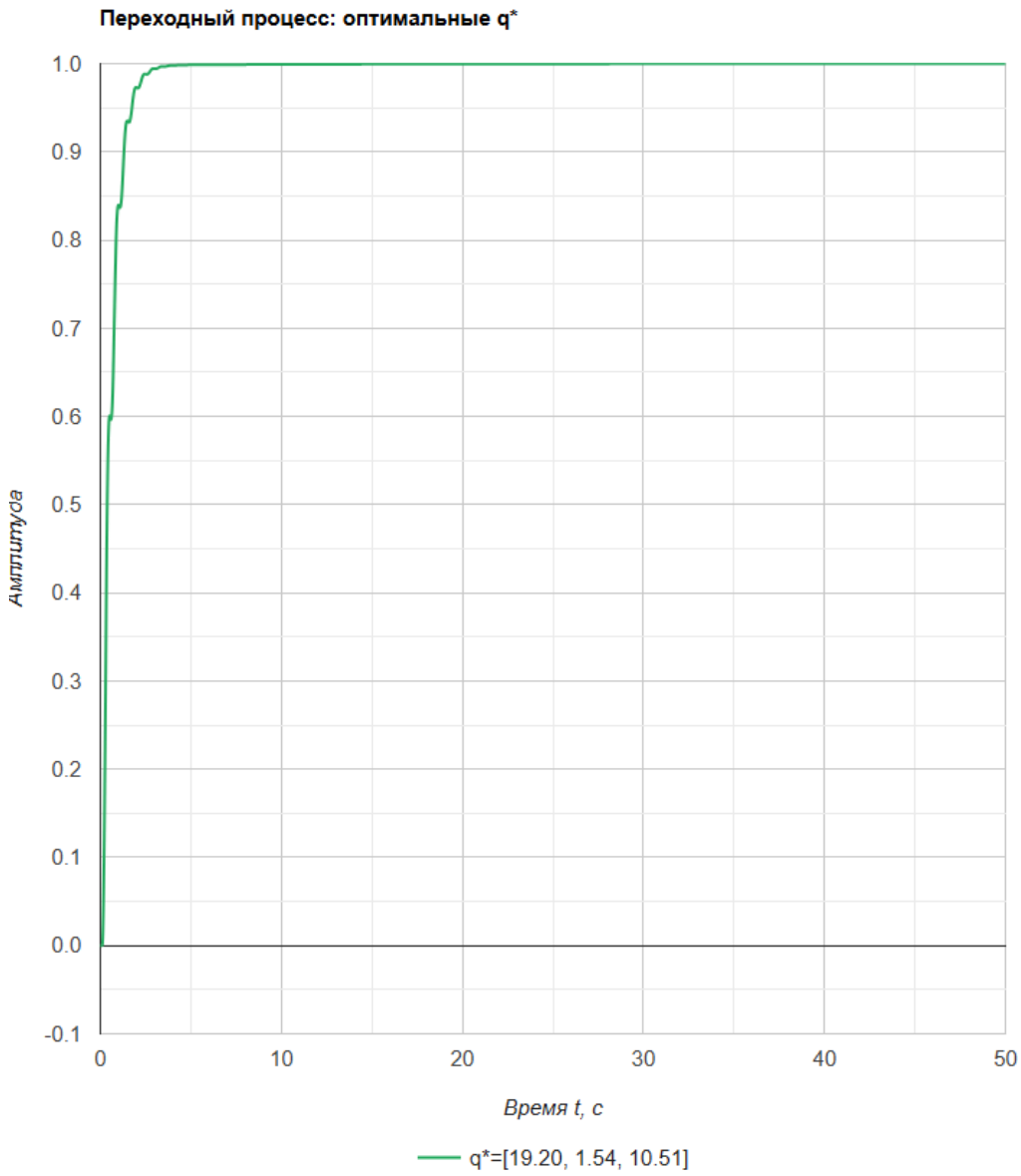

График 18 – Переходный процесс для оптимальных параметров q

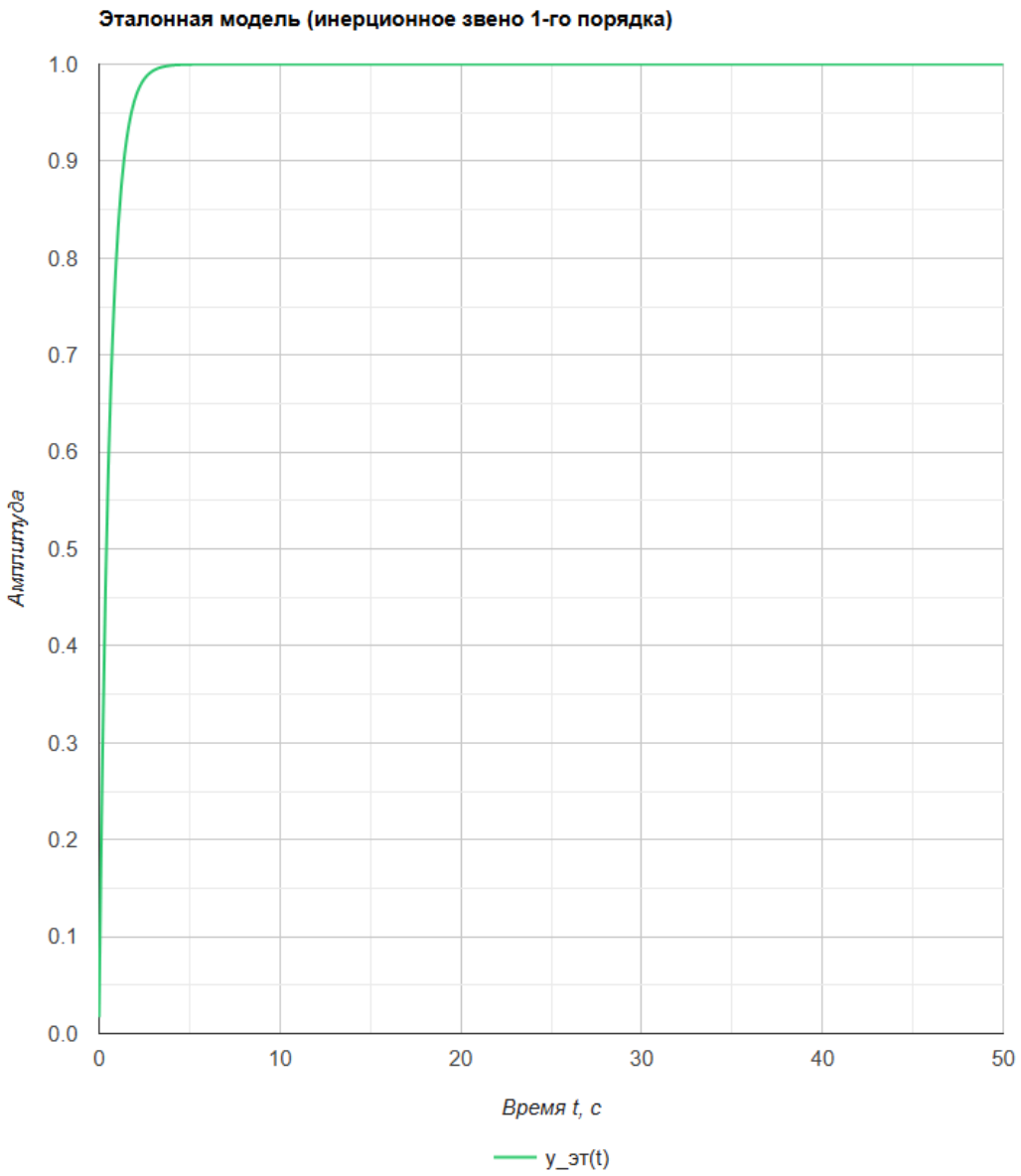

График 19 – Эталонная модель — инерционное звено первого порядка W_эт(p) = k_эт / (T_эт p + 1)

Графики функций чувствительности для различных наборов начальных параметров (до оптимизации):

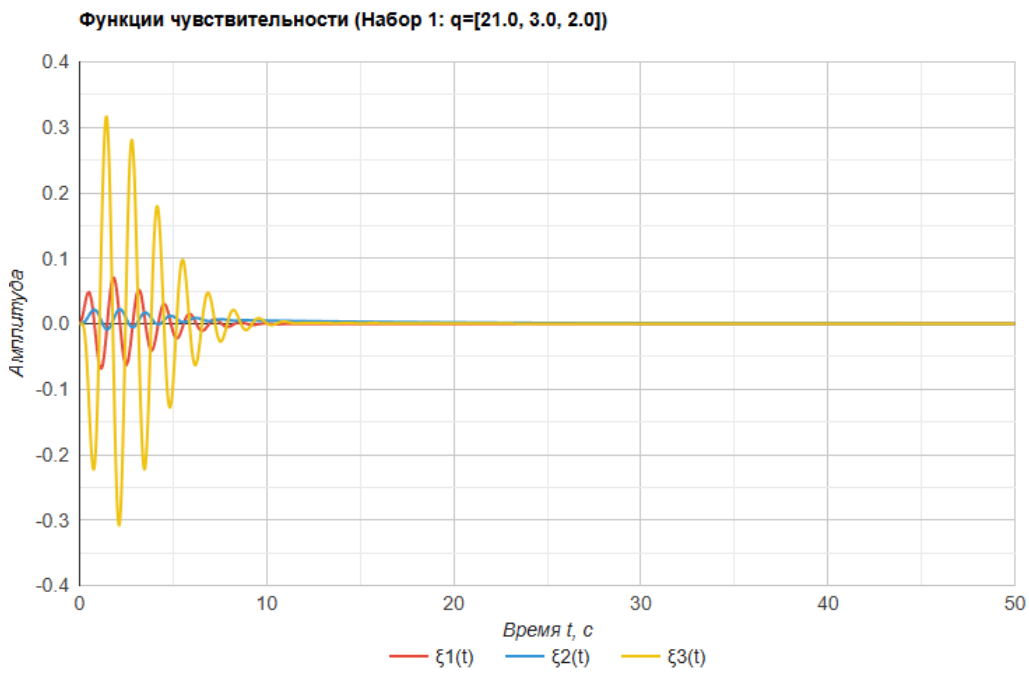

График 20 – Функции чувствительности до оптимизации $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$ для Набора 1: q = [21.0, 3.0, 2.0]

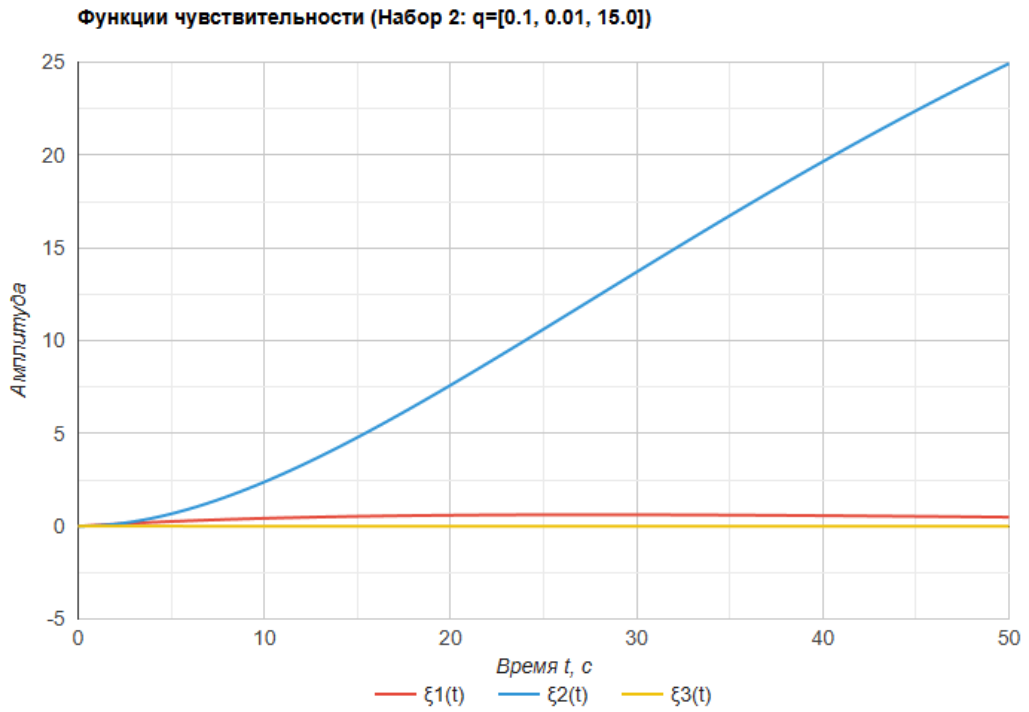

График 21 – Функции чувствительности $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$ для Набора 2: q = [0.1, 0.01, 15.0]

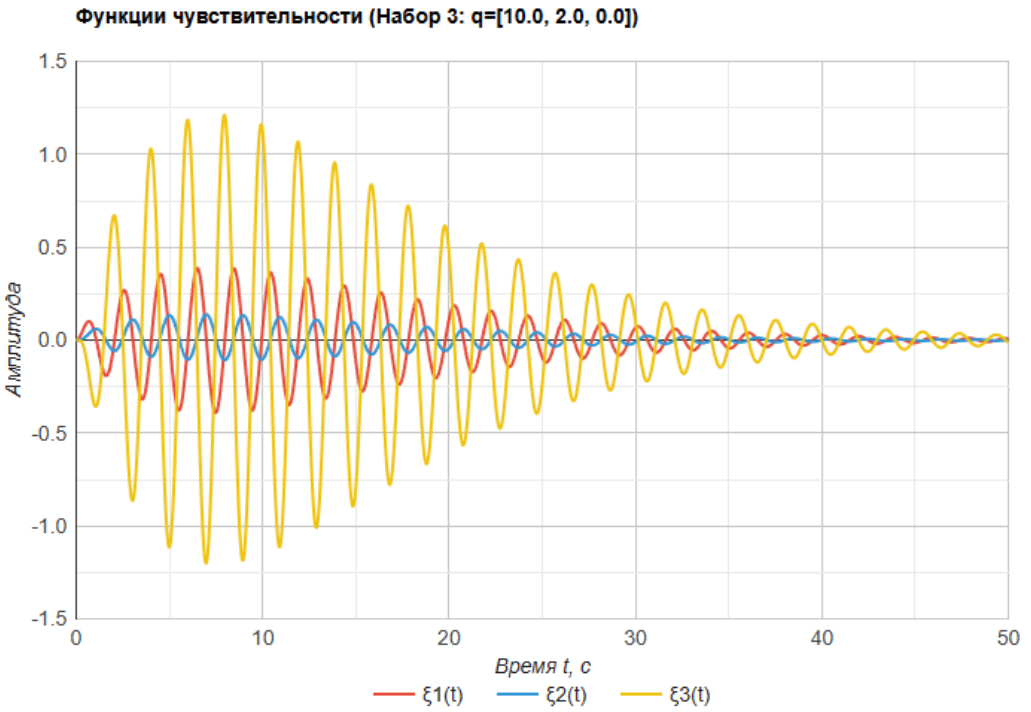

График 22 – Функции чувствительности $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$ для Набора 3: q = [10.0, 2.0, 0.0]

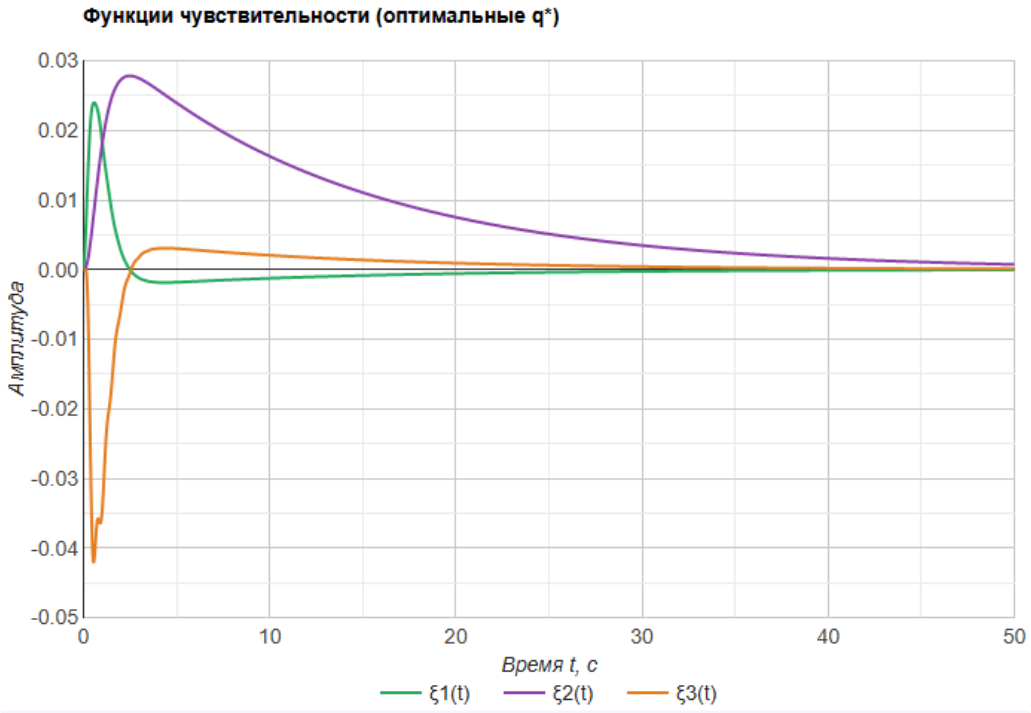

График 23 – Функции чувствительности $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$

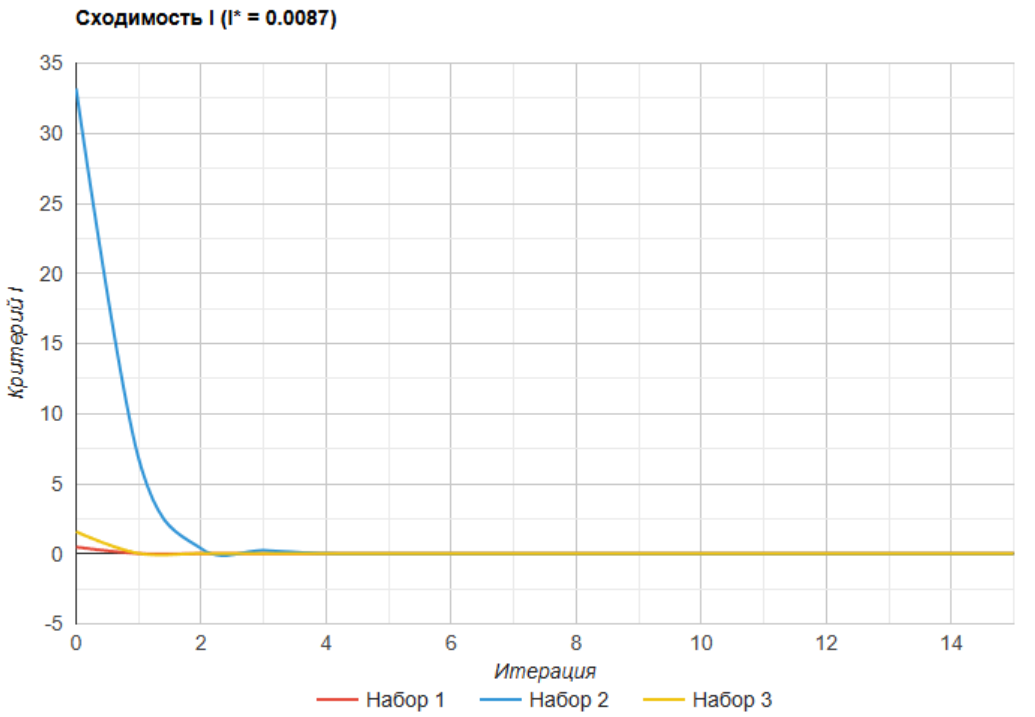

График 24 – Сходимость критерия I для всех 3 наборов

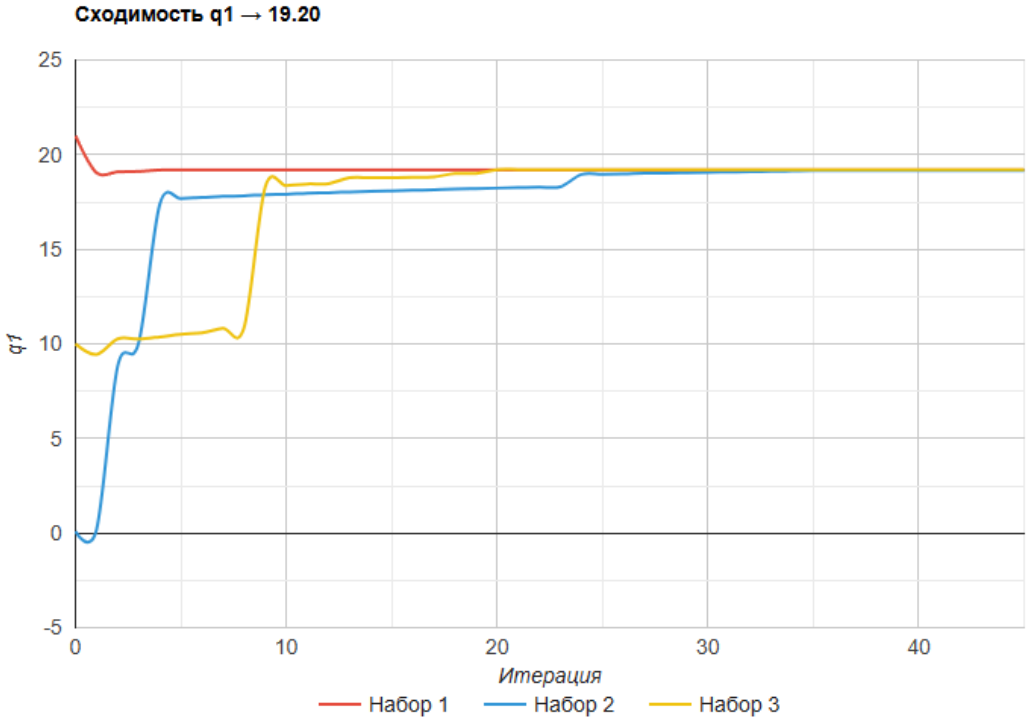

График 25 – Сходимость параметра q1 из 3 различных начальных точек

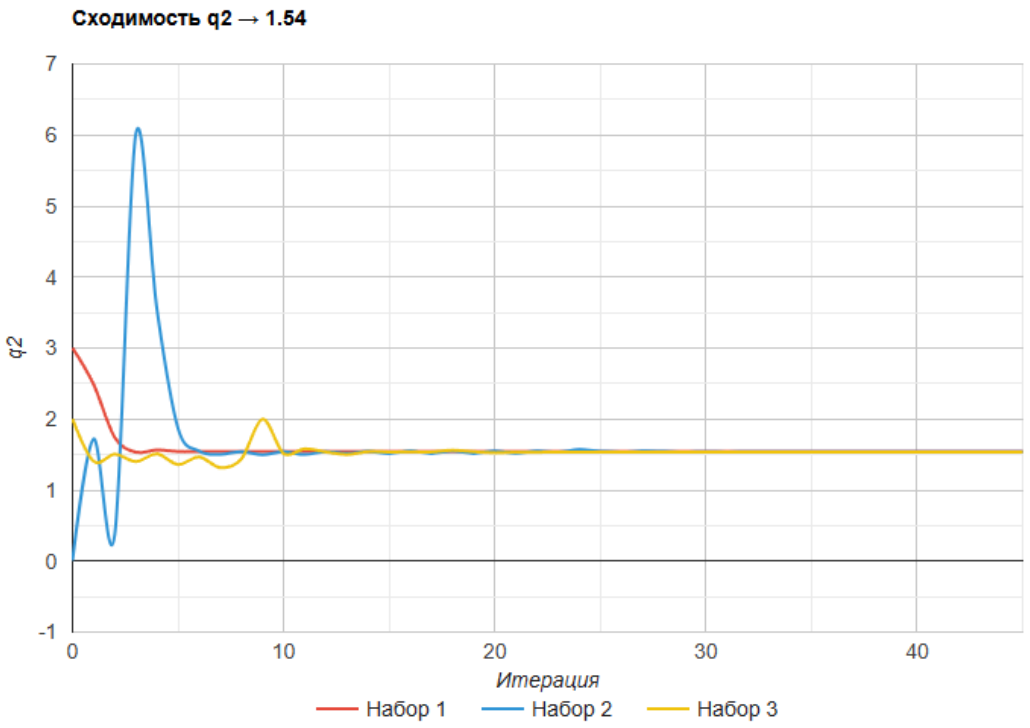

График 26 – Сходимость параметра q2 из 3 различных начальных точек

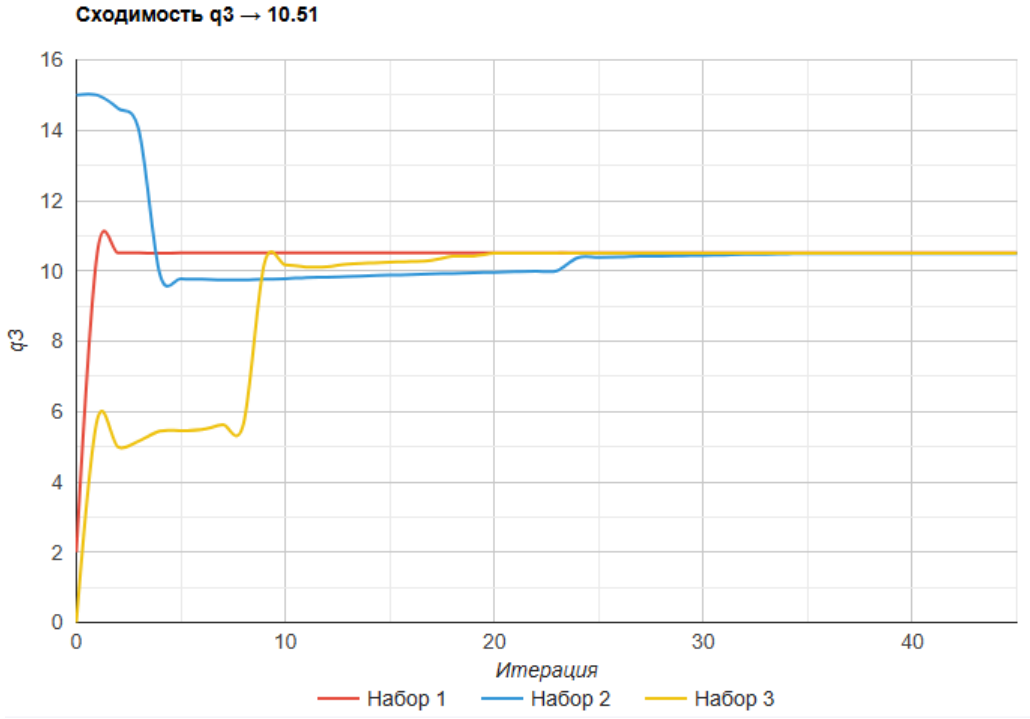

График 27 – Сходимость параметра q3 из 3 различных начальных точек

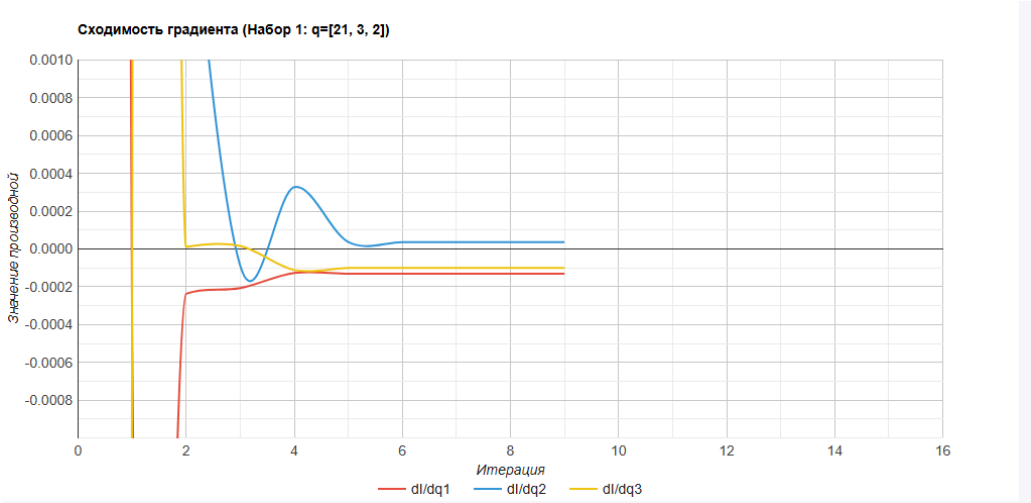

График 28 – Сходимость составляющих градиента к нулю (Набор 1: q=[1.0, 0.0, 0.0])

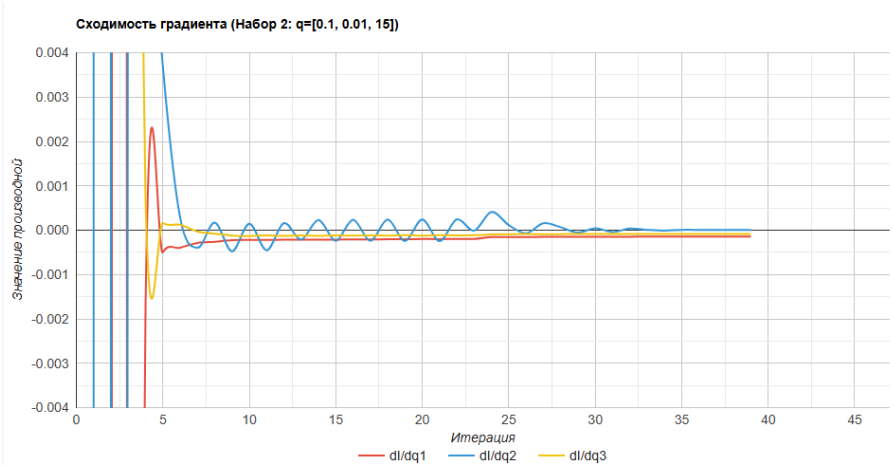

График 29 – Сходимость составляющих градиента к нулю (Набор 2: q=[0.1, 0.01, 7.0])

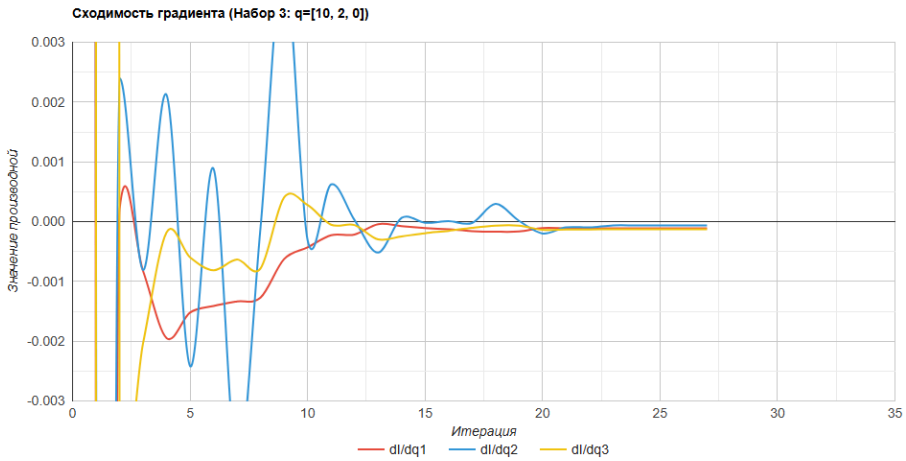

График 30 – Сходимость составляющих градиента к нулю (Набор 3: q=[10.0, 2.0, 0.0])

Результаты исследования

В ходе работы были исследованы три метода параметрической оптимизации ПИД-регулятора:

- Метод 1: Покоординатный спуск
- Метод 2: Градиентный спуск с нормализацией и адаптивным шагом
- Метод 3: Метод наискорейшего спуска с линейным поиском

Параметр	Покоординатный спуск	Градиентный спуск	Наискорейший спуск	Начальный критерий ошибки
Начальные параметры q^0	1.0, 0.0, 0.0	1.0, 0.0, 0.0	1.0, 0.0, 0.0	1.0, 1.0, 1.0
Оптимальные q_1^*	36.64	35.69	19.19	-
Оптимальные q_2^*	4.27	4.04	1.54	-
Оптимальные q_3^*	9.00	7.98	10.51	-
Критерий I^*	0.248	0.250	0.0087	1.1663
Число итераций	50	30	48	-
Сходимость	Медленная	Средняя	Быстрая	-

Преимущества и недостатки методов:

Покоординатный спуск отличается простотой реализации и не требует вычисления градиента, однако обладает медленной сходимостью и требует большого количества итераций.

Градиентный спуск сходится быстрее и использует информацию о градиенте для движения к минимуму, но требует тщательного выбора шага обучения и может осциллировать вблизи оптимума.

Метод наискорейшего спуска показывает наиболее быструю сходимость благодаря линейному поиску вдоль направления антиградиента, однако сложнее в реализации и требует больше вычислений на каждой итерации.

Проверка работоспособности алгоритма наискорейшего спуска: Для подтверждения корректности работы алгоритма оптимизации проведём эксперимент с тремя различными наборами начальных параметров ПИД-регулятора. Независимо от начальных значений, все три набора должны сойтись к одним и тем же оптимальным параметрам, что подтвердит правильность работы метода оптимизации.

Параметр	Набор 1	Набор 2	Набор 3

Начальные параметры q^0	21.0, 3.0, 2.0	0.1, 0.01, 15.0	10.0, 2.0, 0.0
Оптимальные q_1^*	19.20	19.20	19.20
Оптимальные q_2^*	1.54	1.54	1.54
Оптимальные q_3^*	10.51	10.51	10.51
Критерий I^*	0.0087	0.0087	0.0087
Число итераций	9	39	27

Итог исследования: В ходе сравнительного анализа трёх методов оптимизации ПИД-регулятора установлено, что метод наискорейшего спуска демонстрирует наилучшие результаты как по точности (критерий $I^* = 0.0087$ против 0.248–0.250 у других методов), так и по скорости сходимости. Эксперимент с тремя различными наборами начальных параметров подтвердил работоспособность алгоритма: независимо от начальных значений q^0, все наборы сходятся к единым оптимальным параметрам ($q_1^* \approx 19.19$, $q_2^* \approx 1.54$, $q_3^* \approx 10.51$), что подтверждает корректность реализации метода наискорейшего спуска.

Вывод

В ходе выполнения лабораторной работы были изучены и реализованы три метода параметрической оптимизации ПИД-регулятора: покоординатный спуск, градиентный спуск с нормализацией и метод наискорейшего спуска с линейным поиском.

По скорости сходимости методы расположились в следующем порядке (от медленного к быстрому):

- Покоординатный спуск — требует наибольшего количества итераций из-за последовательной оптимизации каждого параметра;
- Градиентный спуск — сходится быстрее благодаря использованию информации о градиенте;
- Наискорейший спуск — показывает наилучшую скорость сходимости за счёт линейного поиска вдоль направления антиградиента.

По точности оптимизации (минимальному значению критерия качества I^*):

- Наискорейший спуск: $I^* = 0.0087$ (наилучший результат);
- Покоординатный спуск: $I^* = 0.248$;
- Градиентный спуск: $I^* = 0.250$.

Метод наискорейшего спуска превзошёл остальные методы по точности, что демонстрирует его высокую эффективность для решения задач параметрической оптимизации ПИД-регуляторов.

Практическая значимость: Результаты работы подтверждают, что выбор метода оптимизации является критическим фактором при настройке систем управления. Для инженерных задач рекомендуется отдавать приоритет методу наискорейшего спуска или алгоритмам второго порядка, так как они обеспечивают наиболее точное приближение к оптимальным параметрам за меньшее число шагов.